

Chapter 2 Data Representation

The mathematical bit pattern for decimal **-5** in an **8-bit architecture** using the **Excess-128** format is 01111011.

Exercise No. 1

What is Excess format? Excess notation shifts the value scale by adding a constant **bias** value to the true number before converting it into unsigned binary. For an m-bit integer, the standard bias is 2^{m-1} .

- **Rule for m=8, Bias = 128:** Calculate Value + 128, then convert that unsigned sum directly into binary.

Step-by-Step Calculation

To find the biased representation (Excess-X), you shift the original value by adding the bias.

- **Identify the bias:** The bias is **128**.
- **Apply the bias:** Add the bias to the target decimal value.
-5+128=123
- **Convert to binary:** Convert the decimal result (**123**) into an 8-bit binary number.

Binary Breakdown for 123

- **128s place:** 0
- **64s place:** 1
- **32s place:** 1
- **16s place:** 1
- **8s place:** 1
- **4s place:** 0
- **2s place:** 1
- **1s place:** 1

Adding the active bits (64 + 32 + 16 + 8 + 2 + 1) yields **123**, which is written in 8 bits as **01111011**.

Exercise 2 Compare the excess format with one’s complement and two’s complement

Step-by-Step Derivation and Explanation for one’s complement and two’s complement

To understand how these patterns are created, we start with the standard 8-bit unsigned representation of positive 5: **00000101**.

1. One's Complement (11111010)

- **How it works:** In one's complement, a negative number is represented by flipping (inverting) every single bit of its positive counterpart.
- **Derivation:**
 - Start with +5: 00000101
 - Invert all bits (0 to 1) and (1 to 0): **11111010**
- **Mathematical Property:** The Most Significant Bit (MSB) acts as the sign bit (1 means negative). The system mathematically yields two representations for zero (00000000 for +0 and 11111111 for -0), which complicates CPU arithmetic logic.

2. Two's Complement (11111011)

- **How it works:** This is the universal standard for integer arithmetic in modern computers. It builds upon one's complement by adding 1 to the inverted result.
- **Derivation:**
 - Take the One's Complement of -5: **11111010**

- **Add 1:** $11111010 + 00000001 = 11111011$

- **Mathematical Property:** The MSB is still the sign bit (1). The mathematical weight of the MSB in an 8-bit system is $-2^7 = -128$.

$$\text{Value} = (-128 \times 1) + 64 + 32 + 16 + 8 + 0 + 2 + 1 = -5$$

This eliminates the duplicate zero problem and allows standard addition hardware to handle subtraction perfectly.

Exercise 3 Question: Explain how Excess-128 handles floating-point exponents?

Excess-128 (also known as **biased-128 representation**) handles floating-point exponents by adding a fixed bias of 128 to the actual exponent value before storing it as a binary number. This shift ensures that all exponents—both positive and negative—are stored as positive integers, eliminating the need for a separate sign bit within the exponent field.

Here is exactly how the system processes, stores, and reads floating-point exponents:

The Mathematical Formula

- **Stored Exponent** = Actual Exponent + 128
- **Actual Exponent** = Stored Exponent - 128

Exercise 4: Why It Is Used

In floating-point math, exponents can be negative (for microscopic fractions) or positive (for massive numbers). Instead of using standard signed binary formats (like two's complement), computer hardware uses a biased notation because it allows for **faster comparisons**. A hardware processor can determine which of two floating-point numbers is larger simply by comparing their stored bits from left to right, without doing complex signed math.

Example 1: Storing a Positive Exponent

If you want to represent a number with an actual exponent of **+5** (e.g. 1.011×2^5)

Steps:

1. **Apply the bias:** $5 + 128 = 133$
2. **Convert to 8-bit binary:** 133 becomes 10000101
3. **Result:** 10000101 is saved in the exponent slot.

Example 2: Storing a Negative Exponent

If you want to represent a number with an actual exponent of **-5** (e.g., 1.011×2^{-5}):

1. **Apply the bias:** $-5 + 128 = 123$
2. **Convert to 8-bit binary:** 123 becomes 01111011
3. **Result:** 01111011 is saved in the exponent slot.

Key Value Ranges

Because Excess-128 typically uses an **8-bit field**, the stored integer values range from 0 to 255:

- **Stored 00000000 (0):** Represents the minimum actual exponent, **-128**
- **Stored 10000000 (128):** Represents an actual exponent of **0**
- **Stored 11111111 (255):** Represents the maximum actual exponent, **+127**

Exercise 5: Calculate and Explain the Given: 25.625 in decimal; Required: Represent the given number in IEEE754 (single precision)

The **IEEE 754 single-precision representation** for the decimal number **25.625** is **0 10001111 100110100000000000000000**

Step by step Solution:

The IEEE 754 single-precision format splits a 32-bit layout into **three distinct components**:

- 1. **Sign bit** (1 bit)
- 2. **Biased Exponent** (8 bits)
- 3. **Fraction / Mantissa** (23 bits)

Step-by-Step Conversion Process

1. Convert the Decimal Number to Binary

Separate the number into its integer and fractional parts:

- **Integer Part (25):**
Divide repeatedly by 2 and track the remainders:
 - 25 div 2 = 12 remainder **1**
 - 12 div 2 = 6 remainder **0**
 - 6 div 2 = 3 remainder **0**
 - 3 div 2 = 1 remainder **1**
 - 1 div 2 = 0 remainder **1**

Reading the remainders from bottom to top gives: $25_{10} = 11001_2$.

- **Fractional Part (0.625):**
Multiply repeatedly by 2 and track the resulting integer parts:
 - $0.625 \times 2 = 1.25 \rightarrow 1$
 - $0.25 \times 2 = 0.50 \rightarrow 0$
 - $0.5 \times 2 = 1.00 \rightarrow 1$
 - Reading the integer parts from top to bottom gives: $0.625_{10} = 0.101_2$
- Combining both parts, $25.625_{10} = 11001.101_2$

2. Normalize the Binary Number

Shift the binary point to the left so there is exactly one non-zero digit to its left, adjusting the exponent of 2 accordingly:

$11001.101_2 = 1.1001101_2 \times 2^4$

- The actual exponent is **4**

3. Determine the Three IEEE 754 Components

- **Sign Bit (1 bit):**
The number is positive, so the sign bit is **0**. (If it were negative, it would be 1).
- **Biased Exponent (8 bits):**
Add the single-precision bias of **127** to the actual exponent:
 $\text{Biased Exponent} = 4 + 127 = 131_{10}$
Convert 131 to an 8-bit binary string: $131 = 128 + 2 + 1 \rightarrow 10000011$.
- **Mantissa / Fraction (23 bits):**
Take the fractional bits to the right following the binary point from the normalized form (1001101).
Pad with trailing zeros to fill the remaining space up to 23 bits:
Mantissa = 10011010000000000000000

Final Bit Layout: 01000011110011010000000000000000

- **Full 32-bit String:** 0 10000011 100110100000000000000000
 - **Hexadecimal Representation:** 0x41CD0000
-

Solution:

Step-by-Step Breakdown

An IEEE 754 single-precision number uses 32 bits, divided into three parts:

- 1. **Sign bit S:** The 1st bit (leftmost).
- 2. **Exponent (E):** The next 8 bits.
- 3. **Mantissa / Fraction (M):** The remaining 23 bits (rightmost).

Splitting the input 11000010011001100000000000000000:

- **Sign bit:** 1
- **Exponent bits:** 10000100
- **Mantissa bits:** 011001100000000000000000

1. Determine the Sign

- The sign bit is 1, which means the final decimal number is **negative** $-1^1 = -1$.

2. Decode the Exponent

- Convert the 8-bit binary exponent 10000100 to decimal:
 $128+0+0+0+0+4+0+0=132_{10}$
- Subtract the single-precision bias of **127** to find the true exponent e:
 $e=132-127 = 5$

3. Decode the Mantissa

- Add the implicit leading 1. to the 23-bit fractional mantissa:
 $1.01100110000000000000000_2$
- Convert the fractional part into decimal:
 - Bit 1 $2^{-1} = 0 \times 0.5 = 0$
 - Bit 2 $2^{-2} = 1 \times 0.25 = 0.25$
 - Bit 3 $2^{-3} = 1 \times 0.125 = 0.125$
 - Bit 4 $2^{-4} = 0 \times 0.0625 = 0$
 - Bit 5 $2^{-5} = 0 \times 0.03125 = 0$
 - Bit 6 $2^{-6} = 1 \times 0.015625 = 0.015625$
 - Bit 7 $2^{-7} = 1 \times 0.0078125 = 0.0078125$
 - Sum of fraction = $0.25 + 0.125 + 0.015625 + 0.0078125 = 0.796875$
- Total value of the significand = $1 + 0.796875 = 1.796875$

4. Calculate Final Value

Use the IEEE 754 formula: $-1^S \times (1 + \text{fraction}) \times 2^e$

- $(-1)^1 \times 1.796875 \times 2^5$
- **= -1 x 1.796875 x 32 = -57.5**